

Guide Détaillé v3 — Sahar

Docker complet + Agent 1 (détection Zephyr) + Agent 4 (RAG) + support Yocto

Ce qui change par rapport à ta version précédente

- L'**Agent 1** ne détecte plus un projet Arduino, mais un projet **Zephyr RTOS** (présence de `prj.conf`, `CMakeLists.txt`).
 - L'**Agent 4 (RAG)** doit maintenant intégrer la documentation Zephyr en plus de Docker/MQTT.
 - Tu apportes un **support ponctuel** à Ameni en semaine 5-6 pour intégrer Docker dans son image Yocto.
 - Le reste (infra Docker complète, CI/CD transféré à Ameni) ne change pas.
-

Semaine 1 — Docker de base + FastAPI + premier LLM

(Identique à ta version précédente : `Dockerfile backend`, `squelette docker-compose.yml`, `test_llm.py`.)

Semaine 2 — Mosquitto

(Identique : ajout du service Mosquitto au `docker-compose.yml`, réseau Docker.)

Semaine 3 — InfluxDB + Grafana (infra)

(Identique : tu livres l'infrastructure, Nour et Ameni y branchent leurs données/dashboards.)

Semaine 4 — Agent 1 : détection Zephyr RTOS

■ `prj.conf` : le fichier de configuration central d'un projet Zephyr, qui active/désactive les fonctionnalités du système (WiFi, MQTT, drivers de capteurs, etc.). ■

`CMakeLists.txt` : le fichier qui décrit comment construire le projet (quels fichiers source compiler), utilisé par l'outil CMake sur lequel s'appuie Zephyr.

```
python
```

```

from git import Repo
import tempfile, os, json
import anthropic

client = anthropic.Anthropic(api_key="TA_CLE_API")

def analyser_depot(url_github: str) -> dict:
    dossier_temp = tempfile.mkdtemp()
    Repo.clone_from(url_github, dossier_temp)

    fichiers_detectes = []
    contenu = ""
    for racine, _, fichiers in os.walk(dossier_temp):
        for f in fichiers:
            if f in ("prj.conf", "CMakeLists.txt") or f.endswith(".conf"):
                fichiers_detectes.append(f)
                chemin = os.path.join(racine, f)
                with open(chemin, "r", errors="ignore") as fichier:
                    contenu += f"\n--- {f} ---\n" + fichier.read()

    prompt = f"""
Voici les fichiers détectés dans un projet embarqué : {fichiers_detectes}
Contenu extrait :
{contenu[:8000]}

Réponds UNIQUEMENT en JSON avec ce format exact :
{{
  "framework": "Zephyr RTOS" ou "Arduino" ou "inconnu",
  "fichiers_detectes": [...],
  "carte_cible": "...",
  "protocoles": [...]
}}
"""

    response = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=500,
        messages=[{"role": "user", "content": prompt}]
    )
    return json.loads(response.content[0].text)

```

Livrable : publie `mock_agent1_output.json` pour Ameni dès que le format est stable :

json

```
{
  "framework": "Zephyr RTOS",
  "fichiers_detectes": ["prj.conf", "CMakeLists.txt"],
  "carte_cible": "esp32",
  "protocoles": ["MQTT", "WiFi"]
}
```

Semaine 5 — Consolidation Docker + démarrage de la base RAG

(Identique à ta version précédente : nettoyage de l'infra, scan Trivy en bonus, début de collecte documentaire pour l'Agent 4.)

Nouveau pour cette version : commence aussi à rassembler la **documentation officielle de Zephyr** (site docs.zephyrproject.org, section "Getting Started" et "Samples") comme source pour ton Agent 4 — c'est le bon moment pour la télécharger/extraire pendant que ta semaine est plus légère.

Semaine 6 — Agent 4 (RAG) + support Docker-dans-Yocto

Partie A — Agent 4 (RAG), logique inchangée mais base documentaire élargie

```
python

import chromadb
from langchain_text_splitters import RecursiveCharacterTextSplitter

client_chroma = chromadb.PersistentClient(path="./rag_db")
collection = client_chroma.get_or_create_collection("docs_techniques")
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)

def indexer_document(chemin_fichier: str, source: str):
    with open(chemin_fichier, "r") as f:
        texte = f.read()
        chunks = splitter.split_text(texte)
        ids = [f"{source}_{i}" for i in range(len(chunks))]
        collection.add(documents=chunks, ids=ids, metadatas=[{"source": source} for _ in chunks])

# Sources à indexer :
indexer_document("docs/erreurs_docker.md", "erreurs_docker")
indexer_document("docs/erreurs_mqtt.md", "erreurs_mqtt")
indexer_document("docs/zephyr_getting_started.md", "zephyr_docs")
indexer_document("docs/erreurs_build_zephyr.md", "erreurs_zephyr") # rempli par Amen
```

```
python
```

```
def demander_a_assistant(question: str) -> str:
    resultats = collection.query(query_texts=[question], n_results=3)
    chunks_pertinents = "\n---\n".join(resultats["documents"][0])
    prompt = f"""
    Extraits de documentation pertinents :
    {chunks_pertinents}

    Question : {question}

    Réponds en te basant sur ces extraits. Si l'information n'y est pas,
    dis-le clairement plutôt que d'inventer.
    """
    response = client.messages.create(
        model="claude-sonnet-4-5", max_tokens=500,
        messages=[{"role": "user", "content": prompt}]
    )
    return response.content[0].text
```

Tester avec un cas réel Zephyr :

```
python
print(demander_a_assistant("L'erreur 'west: command not found' apparaît lors du build,
```

Partie B — Support ponctuel à Ameni (Docker dans Yocto)

■ **Intégrer Docker dans une image Yocto** : ajouter une "recette" BitBake qui installe le moteur Docker à l'intérieur de l'image Linux générée, pour que le Raspberry Pi ait Docker disponible dès son premier démarrage.

Ton rôle ici n'est **pas de maîtriser Yocto toi-même**, mais d'apporter ton expertise Docker :

- Vérifie avec Ameni que la version de Docker Engine choisie pour la recette Yocto est compatible avec tes images (celles de ton `docker-compose.yml`).
- Aide à tester que les conteneurs Mosquitto/InfluxDB/Grafana démarrent correctement une fois l'image Yocto lancée (dans QEMU ou sur le vrai Raspberry Pi).

Livrable : l'Agent 4 répond correctement sur au moins 3 cas d'erreurs réels (Docker + Zephyr) + validation que ton infra Docker tourne dans l'environnement d'Ameni.

Semaine 7 — Intégration finale

Chaîne complète : GitHub → Agent 1 (toi) → Agent 2 (Ameni) → Agent 3 (Nour) → Build Runner (Ameni) → Déploiement sur passerelle (Yocto ou Raspberry Pi OS standard) → Monitoring → Diagnostic RAG (toi).

Ta soupape de sécurité (mise à jour)

1. Sacrifie en premier : le support Yocto de la semaine 6 (Ameni peut avancer seule si tu es débordée par le RAG).
2. Sacrifie ensuite : l'indexation de la doc Zephyr complète dans le RAG — garde au moins les erreurs de build les plus fréquentes.
3. Ne sacrifie jamais : la publication de `mock_agent1_output.json`.

Combine ce document avec le cahier des charges fusionné pour la vue d'ensemble.